

AI · 数字化落地 · 作品集

黄 家 辉

AI数字化落地作品集

从「会规划」到「能落地、可演示、可量化、可复用」

AI 项目经理 · workflow落地工程师 · 智能体开发 · FDE 前线部署工程师

5 年+企业科技服务 · 300+ 项目 · 100+ 服务企业 · 17 个可追溯证据模块

目录

第一章 • 关于我	4
第二章 • 核心亮点	5
第三章 • 目标岗位与能力组合	6
第四章 • FDE 前线部署能力	7
第五章 • 作品集模块 (17 个)	9
01 AI 工具体系 0-1 搭建规划包	11
02 AI 工作流落地标杆 A: 企业智能问答与申报文档 AIGC	13
03 AI 工作流落地标杆 B: 财务 / 行政 / 外贸跟单	15
04 企业级智能体系统 enterprise-agent	17
05 业务数据库与数据监测报告 (SQL + BI/ChatBI)	20
06 RAG 知识库 Demo (真实带数据演示)	22
07 AI 评测与可观测性 (CI 回归 + OTel/Jaeger)	24
08 办公流程自动化 (Open Claw + Python)	26
09 Prompt 工程与评测体系	28
10 培训 / SOP / 推广运营包	30
11 技术底座与运维证据	32
12 项目复盘与量化数据报告	34
13 企业级智能体 MCP Server 三件套 (开源)	36
14 dsh-bridge: 让 Marvis 调用本机 DSH (开源)	38
DeepSeek Harness 本机插件三件套 (开源)	41
DSH 运维工具箱: 升级自检 / 插件兼容预检 / 生态全量调研 (开源)	43

知识指南：把「先检索再作答」写成可复用的 规程（开源）	45
第六章·行业定制规划能力·行业定制样本	47
第七章·现场可演示清单	48
第八章·诚信说明与口径	49

壹 关于我

从「会规划」到「能落地、可演示、可量化、可复用」

5年+

企业科技服务与项目全流程经验

300+

政府/企业项目交付

100+

服务企业客户

17项

可追溯、可演示的深度作品

一句话介绍

我是**黄家辉**，5年+企业科技服务与项目全流程经验，坐标广州，计算机应用技术背景，软考中级，信息系统项目管理师备考中。

目标岗位

AI 项目经理 / AI 工作流落地工程师 / AI 智能体开发工程师 / FDE 前线部署工程师。

我的定位

我擅长把 AI 从「概念」落到「业务现场」——从需求诊断、方案设计、端到端交付到现场培训，全程打通最后一公里。本作品集是一份完整的**证据链**，每一项都可追溯、可验证、可演示。

我的职业内核

我的职业内核是 **FDE (Forward Deployed Engineer, 前线部署工程师)** 画像：5年+深入客户企业一线，从需求诊断 → 方案设计 → 端到端交付 → 现场实施指导 → 把客户团队教会，全程「把方案真正落到客户手里」，并用高企通过率、最快1周交付、100+项目 / 服务企业100+等量化ROI说话。

贰 核心亮点

六维能力，一张表看清

维度	内容
真实项目	24 家（含复审）单年高企通过 · 1 周最快交付 · 300+ 项目可研
工程落地	6 层智能体架构 · 93.3% RAG 检索准确率 · 35 项测试通过
开源贡献	3 个 MCP Server + 1 个跨应用桥接 MCP Server 开源 · CI 全绿
数据驱动	每个项目都有前后对比量化数据，全部标注口径
FDE 能力	完整能力映射表 + 2 则实战故事（业务型 + 工程型）
能带队	5 年+客户现场交付 · 从诊断到培训全程主导

* 成效类数字凡涉测算，均标注「测算模型 / 示例口径」，实际以落地数据为准。

叁 目标岗位与能力组合

针对不同目标岗位，分别组织对应的工作证据组合

AI 项目经理

能从公司层面搭建 AI 工具体系、跨部门推进落地，并以 ROI 量化价值，而非停留在单个工具的使用。

对应作品：AI 工具体系 0-1 规划包 + 跨部门推进证据（RACI/周报/验收）+ ROI 报告 + 培训推广包

AI 工作流落地工程师

能将 AI 工作流真正跑通、推动业务人员用起来，并基于反馈持续迭代优化。

对应作品：2-3 个「AI 先做一遍、人只审核」流程 + 操作手册 + 培训反馈 + Prompt 评测体系

AI 项目负责人（电商 / 增长）

具备数据看板、流程自动化与团队带教的通用能力，可迁移至电商多平台数据打通与经营看板场景（下含自建 demo 与示例口径，非电商客户实绩，慎用于强电商背景要求）。

对应作品：数据监测报告 + BI 看板能力 + Open Claw/Python 自动化 + 多维表「AI 先做人审核」+ 带教 SOP

AI 智能体开发工程师

具备工程化能力：架构、RAG、工具调用、记忆、容错、部署，既能搭建也能保障稳定运行。

对应作品：企业级智能体系统 enterprise-agent（本地工程）+ 六层架构 + LangGraph 编排/RAG/双视野记忆/工具调用 + Docker/K8s 部署 + 容错重试熔断

FDE 全栈前线部署工程师

职业内核即 FDE 画像：5 年+深入客户企业一线，从需求诊断到端到端交付、现场实施指导、把客户团队教会，并用高企通过率、最快 1 周交付、100+ 项目 / 服务企业 100+ 等量化 ROI。详见第四章「FDE 能力映射」与两则实战故事。

对应作品：高企认定现场交付（业务型 FDE）+ enterprise-agent 本地部署（工程型 FDE）+ 培训/SOP 打通最后一公里 + 数据监测报告

肆 FDE 前线部署能力

深入一线、端到端交付、把方案真正落到客户手里

FDE 的核心不是「会做技术」，而是深入客户现场、把方案端到端落地、并教会客户团队用起来。下表把 FDE 的五项核心要求逐条映射到本人真实经历与作品集证据。

一句话定位：本人 5 年+做的是「科技服务/项目申报的现场交付」——本质就是 FDE 工作法：进客户门、诊断业务、端到端交付、现场培训、用通过率与周期量化结果。AI 与工程能力（enterprise-agent、Coze、RAG、部署）是把这套方法论从「人力密集」升级为「AI 提效」的新引擎。

FDE 能力映射表

FDE 核心要求	本人真实经历对应	作品集证据
深入一线 · 需求诊断	面向客户企业做需求调研、痛点诊断与可行性分析，累计覆盖 100+ 家企业	模块 1 规划包 · 模块 12 复盘
端到端交付	从方案编制、材料合规到申报交付一条龙，最快 1 周完成签单到申报交付	高企认定实战故事（下）
业务沟通 · 跨部门	协同营销做客户诊断与谈判支持，联动财务/技术完成研发费用归集与合规自查	模块 10 培训/带教 · 模块 3 财务合规
快速搭原型 · 现场部署	在本地/资源受限环境用 Docker Compose 快速起 enterprise-agent、现场调试降级容错	模块 4 · 模块 11 现场部署
ROI 量化 · 数据说话	自建项目信息数据库，输出《项目监测报告》，减少团队约 40% 重复工作	模块 5 数据监测 · 模块 12 复盘

实战故事一 · 业务型 FDE

业务型 FDE · 真实项目

高新技术企业认定 · 端到端现场交付

客户痛点：科技型企业想拿高企资质享税收优惠，但不懂研发费用归集、知识产权与材料合规，自己报大概率被否。

深入现场诊断：进企业盘研发项目、知识产权、财务口径与科技人员占比，定位申报缺口。

端到端方案：制定申报路线 → 指导研发费用辅助账归集 → 编制技术方案与申报材料 → 组织合规性自查 → 跟进审批。

打通最后一公里：给企业财务/技术团队做政策与系统操作培训，让客户具备后续自主申报能力。

结果：单年最高通过 24 家（含复审）、单年通过项目 60+，最快 1 周完成从签单到申报交付。

（真实业务经验；具体企业名与年度数据面试时可脱敏提供）

实战故事二 · 工程型 FDE

工程型 FDE · 真实工程

enterprise-agent · 资源受限环境现场部署

业务痛点：要在一台普通机器上跑通一套企业智能体（编排/RAG/记忆/工具调用），不能依赖云端与专职运维。

快速搭原型：用 Docker Compose 一键拉起 agent-api + Redis + pgvector，本地即可演示，绕开复杂环境依赖。

现场调试与救火：用 OpenAI 兼容接口切本地 Ollama 模型、配置降级备用模型，解决启动依赖、模型不可用与超时问题。

稳定交付：内置重试/熔断/调用次数限制与 FastAPI 健康检查，保证现场长时间演示不掉线。

结果：本地完整跑通并支持现场演示，工程文档齐全，已开源。

github.com/Jav1es/enterprise-agents

FDE 交付物模板

把 FDE 工作法沉淀为可直接复用的交付物——从「客户发现」到「工程评测」的两份标准模板，均为真实内容落地。

- **Discovery Brief · 高企认定客户发现简报：**以高企认定申报服务为背景，用 FDE 工作法（进客户企业 → 诊断研发费用归集缺口 → 知产与评分诊断 → 形成可验证交付范围）产出的客户发现简报，数据取自真实客户台账并已脱敏。
- **Eval Plan · enterprise-agent 评测计划：**针对本地企业级智能体工程 enterprise-agent 的评测计划，定义评测数据集、指标合同、RAG 五层与工具调用专项、发布门槛与生产监控闭环。

伍 作品集模块（17 个）

每个模块含：业务痛点、我的角色、方案架构、工具栈、实施步骤、关键难点、量化测算、可复用资产、证明链接与复盘。量化数据凡涉成效均标注为测算模型 / 示例口径，实际以真实落地数据为准。

模块总览

#	模块	类型	核心亮点
01	AI 工具体系 0-1 搭建规划包	方案规划	300+ 项目可研/立项经验 · 8 大业务域覆盖 · 6 个月分阶段路线图
02	AI 工作流落地标杆 A：企业智能问答与申报文档 AIGC	自建 Demo	≥50% 生成效率提升测算 · ≥85% 审核通过率目标 · AI 先做+人工只审 2 层流
03	AI 工作流落地标杆 B：财务 / 行政 / 外贸跟单	已交付	24 家单年高企认定通过（含复审） · 70% 人工审核量下降测算 · 92% 准确率目标
04	企业级智能体系统 enterprise-agent	个人开源	6 层企业级智能体架构 · 70% HPA 扩缩容 CPU 阈值 · 5 次熔断器连续失败阈值
05	业务数据库与数据监测报告（SQL 报表 + BI/ChatBI）	已交付	约 40% 减少团队重复工作 · 1 套自建业务数据库 · SQL 报表固化/存储过程
06	RAG 知识库 Demo（真实带数据演示）	自建 Demo	93.3% 检索准确率@3（30 题分层评测） · 73.3%@1 · 40 条示例制度文档
07	AI 评测与可观测性（CI 回归 + OTel/Jaeger）	自建 Demo	8 个单会话 Jaeger trace span · 6 条 CI 每次 push 自动评测 · ≥60% CI 断言阈值
08	办公流程自动化（Open Claw + Python 脚本）	自建 Demo	跨系统重复流程脚本自动化 · 异常告警+人工复核兜底 · SOP 流程沉淀
09	Prompt 工程与评测体系	自建 Demo	A/B 对比测试驱动优化 · JSON Schema 输出强校验 · Prompt 结构化模板化
10	培训 / SOP / 推广运营包	已交付	70% 自助解决率目标 · 反馈收集→处理→迭代闭环 · 带教培训 SOP 体系
11	技术底座与运维证据	自建 Demo	Docker Compose 一键起服务 · 容错/重试/降级/熔断高可用 · 日志+告警+压测监控
12	项目复盘与量化数据报告	已交付	300+ 政府/企业项目交付 · 24 家单年高企通过（含复审） · 100+ 服务企业客户
13	企业级智能体 MCP Server 三件套（开源）	已交付	3 个 MCP Server 开源 · 35 项 pytest 用例全通过 · 100% 通过率

14	dsh-bridge: 让 Marvis 调用本机 DSH (开源)	开源工具	零依赖手写 MCP 协议 (3 个工具) · 241 技能 × 77 MCP 工具本地检索 · CI 五步全绿
----	------------------------------------	------	--

01 AI 工具体系 0-1 搭建规划包

方案规划

对应 JD：AI 项目经理 / AI 项目负责人

300+

项目可研/立项经验

8

业务域覆盖（8大场景）

6个月

分阶段路线图

企业AI用得乱、没规划。我像做某高分子新材料企业规划那样，先盘场景、用价值×难度排优先级、算ROI给老板看——300+项目可研立项练出来的规划，不是PPT。

业务痛点：公司各部门零散使用 AI 工具，无统一规划、无选型标准、无 ROI 衡量，重复采购、效果不可控。

我的角色：本人具备体系化规划能力：已独立完成某高分子新材料企业信息化总体规划，并累计编制 300+ 政府/企业项目的可行性研究与立项方案；本模块是一套企业级 AI 工具体系 0-1 规划的方法与规划包（业务盘点、选型标准、ROI、路线图与跨部门推进机制）。

方案架构

业务场景盘点（营销/推广/内容/客服/财务/行政/外贸/电商）→ 价值×难度优先级矩阵 → 工具选型对比 → 6 个月路线图 → 成本预算与 ROI 测算 → 跨部门推进机制（RACI/周报/验收标准）。

工具栈

ChatGPT

豆包

Gemini

Midjourney

剪映

即梦

Coze

Dify

n8n

影刀

钉钉千问

实施步骤

1. 盘点 8 大业务场景的 AI 应用机会
2. 用价值×难度矩阵排优先级
3. 对比主流工具（能力/成本/合规/上手难度）
4. 编制 6 个月分阶段路线图
5. 测算工具费+人力+预计节省人天，输出 ROI
6. 设计 RACI 职责矩阵、周报模板与验收标准

关键难点与解决

难点：业务部门需求分散、对 AI 期望不切实际。**解决：**先选 2-3 个高频痛点场景做试点，用可见成效建立信任，再全面推广；用 ROI 测算争取管理层预算支持。

量化测算

真实样本	某高分子新材料企业信息化总体规划+ 300+ 项目可研/立项
覆盖范围	规划包覆盖 8 大业务域 + 工具选型对比 + ROI 测算
优先级方法	价值×难度矩阵定优先级

* 测算模型 / 示例口径，实际以落地数据为准

可复用资产

- 某高分子新材料企业信息化总体规划（真实）
- 工具选型对比表
- 规划路线图
- ROI 测算表
- RACI 推进机制模板

复盘

如果重做，本人会更早引入业务骨干共创场景清单，避免规划与一线脱节；并建立工具使用数据埋点，让 ROI 从「测算」走向「实测」。

02 AI 工作流落地标杆 A：企业智能问答与申报文档 AIGC

[自建 Demo](#)

对应 JD：AI 工作流落地工程师 / FDE / AI 项目负责人

≥50%

生成效率提升测算（示例）

≥85%

审核通过率目标（示例口径）

2层

AI先做+人工只审

政策问答、材料预审、文档编写重复又易错。我用Coze+多维表搭出AI先做、人只审的双层流，demo已跑通方法，效率翻倍、可迁到任意业务。

业务痛点：企业科技服务中，政策咨询、申报材料合规预审、文档与内容编写重复度高、依赖人工，政策更新快、易出错。

我的角色：本人基于 Coze / 本地大模型搭建「AI 先做一遍、人只审核」的工作流（自建 demo 验证方法论，可迁移至项目申报、企业客服与内容场景），配合多维表做人工审核台。

方案架构

输入（政策/材料/需求）→ AI 分类与信息抽取 → 大模型生成草稿/回复 → 多维表人工审核台 → 采纳发布/归档；批量文档处理与数据清理由 Python 承担。

工具栈

[Coze](#)[Dify](#)[n8n](#)[飞书/钉钉多维表](#)[大模型 API](#)[Python](#)

实施步骤

1. 梳理企业问答与文档场景全流程与痛点
2. 设计 AI 预处理 + 人工审核的双层流程
3. 在 Coze/Dify 编排分类、生成、审核节点
4. 用多维表搭建「AI 先做、人审核」工作台
5. 小范围试点 → 收集反馈 → 迭代 Prompt 与流程
6. 输出操作手册并培训业务人员

关键难点与解决

难点：AI 生成内容质量不稳、合规要求高。解决：Prompt 模板 + 输出校验规则 + 人工审核兜底，把「抽卡」变为「可控生产线」；关键结论人工确认。

量化测算

审核量下降	自建 demo 验证：AI 预处理后人工审核量大幅下降（示例：千条级 → 数百条审核）
审核通过率	目标 ≥ 85%（示例口径）
生成效率	内容/文档生成效率提升测算 ≥ 50%

* 测算模型 / 示例口径，实际以落地数据为准

可复用资产

- 问答/文档 workflow 配置
- Prompt 模板库
- 人工审核台模板
- 操作手册

证明链接

- workflow 运行截图
- 数据对比表
- 操作手册
- Prompt 模板库
- 人工审核台模板

复盘

如果重做，本人会把「真实业务数据回流闭环」做得更早，让生成质量由采纳率与错误率数据驱动迭代，而非仅靠经验调 Prompt。

03 AI 工作流落地标杆 B：财务 / 行政 / 外贸跟单

已交付

对应 JD：AI 工作流落地工程师 / AI 项目负责人

24家

单年高企认定通过（含复审）

70%

人工审核量下降测算（示例）

92%

准确率目标（示例口径）

高企和加计扣除卡在研发费用与财务口径。我做过研发费用归集、技术鉴定、单年24家（含复审）高企通过，能把财务和技术口径拉齐、合规自查到点。

业务痛点：发票报销人工逐条审核慢、易错；外贸询盘分类、报价、跟单、报关资料检查靠人工；行政重复报表汇总耗时。

我的角色：我设计 RPA + AI 审核的财务/行政/外贸工作流，把「人工逐条处理」改为「AI 先做一遍、人工只审异常」，并输出 SOP 与异常清单。

方案架构

发票/报销：OCR 识别 → 规则校验 → 大模型合规判断 → 异常表 → 人工复核；外贸：询盘分类 → 报价生成 → 跟单提醒 → 报关资料检查。

工具栈

影刀

实在智能

UiBot

钉钉 RPA

Coze

Dify

n8n

OCR

实施步骤

1. 盘点财务/行政/外贸高频重复流程
2. 设计 OCR+规则+大模型的三层校验流程
3. 用 RPA 打通系统间数据搬运
4. 建立异常清单与人工复核机制
5. 试点 → 统计准确率与人工审核量下降 → 迭代
6. 输出 SOP 与培训材料

关键难点与解决

难点：财务合规要求高、AI 误判风险大。解决：关键节点（金额、税率、合规）保留人工把关，AI 只做预筛与提示；用规则引擎兜底大模型的不确定性。

量化测算

审核量下降	示例测算：人工审核量下降约 70%
准确率目标	≥ 92%（示例口径）
节省工时	测算 ≥ 2 人天/周

* 测算模型 / 示例口径，实际以落地数据为准

可复用资产

- RPA+AI 审核流程配置
- 异常清单模板
- SOP 文档
- 校验规则库

复盘

如果重做，我会先做更细的错误分类统计，针对高频错误类型专门优化 Prompt 和规则，而不是泛泛调优。

04 企业级智能体系统（本地开发工程 enterprise-agent）

[个人开源](#)

对应 JD：AI 智能体开发工程师

6层

企业级智能体架构

70%

HPA扩缩容CPU阈值

5次

熔断器连续失败阈值

通用聊天机器人接不了企业系统。我自建enterprise-agent：LangGraph编排+RAG+双视野记忆+工具调用，Docker一键本地起服务，能现场演示。

业务痛点：通用聊天机器人无法对接企业系统、无持久记忆、无工具调用、输出不可控、故障无兜底，难以支撑真实企业场景。

我的角色：本人独立设计并开发了一套面向企业场景的本地智能体系统，具备 workflow 编排、工具调用、记忆管理、RAG 知识库四大核心能力，通过标准化接入层对接 ERP/CRM/OA/钉钉/企微，内置容错重试与降级机制，支持 Docker Compose 本地部署与 K8s 集群部署。

方案架构（六层）

接入层（钉钉 / 企微 / OA/ERP/CRM Channel）→ 编排层（LangGraph StateGraph：Router→Planner→Skill→Tool→Reviewer）→ 工具层（Native Tool Calling / MCP 协议 / HTTP Connector）→ 记忆层（短期 Redis + 长期 PostgreSQL 事实三元组）→ 知识层（ChromaDB/Milvus, Dense+BM25→RRF→Cross-Encoder 重排）→ LLM 推理层（DeepSeek/Qwen/Ollama 本地模型, OpenAI 兼容接口）。

工具栈

Python 3.11+

FastAPI

LangGraph

LangChain

Pydantic v2

ChromaDB/Milvus

Redis

PostgreSQL

SQLAlchemy

MCP SDK

Docker

Helm/K8s

Pytest

GitHub Actions CI

实施步骤

- 编排层：**用 LangGraph StateGraph 构建 Router→Planner→Skill→Tool→Reviewer 五阶段链路，Pydantic 结构化输出衔接各阶段，支持多步工具调用循环
- 记忆层：**双视野记忆——短期 Redis（会话状态 TTL 30 分钟、滚动摘要、上下文预算控制）+ 长期 PostgreSQL（事实三元组持久化、按访问频率自动提升、按时间修剪、去重与冲突检测）

- 知识层 RAG**: 语义边界智能切分 (段落→换行→句子, chunk 500/overlap 50) → Dense+BM25 混合检索 → RRF 融合 → Cross-Encoder (bge-reranker-base) 重排 → 生成附引用来源
- 工具层与接入**: Pydantic 严格 Schema 定义工具输入输出, UnifiedMessage 统一多平台消息模型, 企微经 wecom-aibot-python-sdk WebSocket 长连接, MCP 系统经 MultiServerMCPClient 标准化工具发现
- 容错与稳定性**: 错误分类处理 (瞬态→指数退避重试、LLM 可恢复→ToolMessage 反馈、Provider 宕机→降级备用模型、过度调用→调用次数限制) + 熔断器 (连续失败 5 次 Open→恢复超时 Half-Open 试探)
- 部署与服务**: Docker Compose 编排 agent-api/redis/pgvector, K8s Helm 部署 (HPA CPU 70% 自动扩缩容、Liveness/Readiness 探针) ; FastAPI 提供同步 /v1/chat 与 SSE 流式 /v1/chat/stream 接口、Swagger UI 文档; Locust 压测 TTFT/P95/P99/QPS/错误率
- 质量与工程化**: 为核心模块编写 pytest 单元测试, GitHub Actions CI 每次推送自动跑 lint + 测试作为合并门禁; 提交前用 check_sensitive.sh 扫描硬编码密钥/Token, 保证源码可公开而不泄露凭证

关键难点与解决

难点: 多阶段智能体协作的稳定性、幻觉控制与故障兜底。解决: 用 Pydantic 结构化输出约束各阶段格式、RAG 混合检索+重排降低幻觉并附引用来源、中间件层做重试/降级/熔断/调用次数限制防死循环, 关键输出经 Reviewer 节点校验。

工程化与容量基线 (本次新增)

补齐了此前只有文档描述、没有可验证产物的部分。三项均为本机实测:

K8s 部署	完整 Helm Chart (values + 7 模板 + 双指标 HPA CPU 70%/内存 80% + /health 双探针 + 非 root 加固 + 密钥强制走 Secret, 缺 Secret 直接拒绝渲染) ; helm lint 通过 + 17 项自动校验全过
容量基线	Locust 阶梯压测 10/20/50/100 并发, 合计 2535 次请求 0 失败; 吞吐饱和拐点落在 50→100 并发之间 (增幅由 +101.3% 骤降到 +39.8%) , 单实例舒适区约 50 并发 / 28 req/s
延迟分解	真实 LLM 调用 5 次采样, 首 token 中位 789.6 ms、占总延迟 61.3%, 据此判定瓶颈在模型侧而非编排代码; SSE 流式把响应头前置到 11 ms, 让用户无需等待首 token

量化测算

记忆	短期记忆 TTL 30 分钟, 长期记忆事实三元组持久化
熔断阈值	连续失败 5 次触发 Open→Half-Open
扩缩容	HPA 自动扩缩容 CPU 阈值 70%
RAG 参数	chunk 500 / overlap 50, Dense+BM25→RRF→重排

压测维度	TTFT / P95-P99 延迟 / QPS / 错误率
------	-------------------------------

* 测算模型 / 示例口径，实际以落地数据为准

可复用资产

六层架构设计文档

LangGraph 工作流编排代码

双视野记忆模块

RAG 混合检索+重排模块

容错/降级/熔断中间件

Docker Compose + Helm Chart

FastAPI 服务与 Swagger 文档

Locust 压测脚本

pytest 单元测试

GitHub Actions CI

敏感信息扫描脚本

证明链接

《企业级智能体系统开发文档》

本地代码工程（可现场演示）

github.com/Jav1es/enterprise-agents

RAG 检索演示-真实运行截图

RAG 评测报告

复盘

这一版已把当初复盘里说的两件事落地：接入了 OpenTelemetry 全链路可观测性（日志/追踪/指标），并把 RAG 评测集纳入 CI 做回归。后续我已把评测集从 6 题扩到 30 题分层（@3 准确率 93.3%），实跑 Cross-Encoder 重排对照后发现当前规模下增量为 0——因为混合检索已把答案放进 Top-3，而重排只能重排不能召回；据此改为可开关并写明启用条件，同时发现 RRF 分数不可用作拒答阈值（答得上与答不上的题分数完全重叠），改用强制带条款引用来防幻觉。

05 业务数据库与数据监测报告（SQL 报表 + BI/ChatBI）

已交付

对应 JD：AI 项目负责人（数据）/ FDE

约40%

减少团队重复工作

1套

自建业务数据库

SQL

报表固化/存储过程

项目数据散在Excel、决策没数。我自建业务库、用SQL固化报表、定期出《项目监测报告》，帮团队砍掉40%以上重复活。

业务痛点：项目与业务数据散落在 Excel 与纸质材料，报表靠人工汇总，管理者看不到实时进度与成效，决策缺数据支撑。

我的角色：本人自建项目信息数据库（业务知识库雏形），用 MySQL 存储、SQL/存储过程开发报表，采集执行数据并定期输出《项目监测报告》，以数据驱动方案迭代与管理决策；该数据方法论可迁移到电商经营看板与 ChatBI 自然语言问数。

方案架构

数据采集（项目立项/进度/材料/财务归集）→ MySQL 业务数据库 → SQL/存储过程查询与报表 → 《项目监测报告》→ 管理决策；延伸：ETL 入仓 → 指标体系 → BI 看板 → ChatBI 自然语言问数。

工具栈

MySQL

SQL / 存储过程

Python

数据分析/建模

FineBI/PowerBI（方法）

钉钉智能报表

实施步骤

1. 盘点数据来源与统计口径
2. 建库并统一字段与主数据口径
3. 用 SQL/存储过程固化常用报表
4. 定期采集数据并输出《项目监测报告》
5. 延伸：ETL 入仓 + 指标体系 + BI 看板
6. 延伸：接入 ChatBI 自然语言问数

关键难点与解决

难点：数据来源分散、口径不一。解决：先统一数据字段与口径建结构化库，用 SQL 固化报表，避免「同名不同义」与人工汇总出错。

量化测算

重复工作	减少团队约 40% 重复工作
自建数据库	自建业务数据库并定期输出《项目监测报告》
核心度量	报表自动化率 / 数据入仓率（示例口径）

* 测算模型 / 示例口径，实际以落地数据为准

可复用资产

- 项目信息数据库结构
- SQL 报表 / 存储过程
- 《项目监测报告》模板
- 指标体系与指标字典

复盘

如果重做，本人会把数据质量监控做进看板，源数据异常自动预警，并把 ChatBI 问数尽早接入，让「看板好看且数据可信」。

06 RAG 知识库 Demo (enterprise-agent 真实带数据演示)

自建 Demo

对应 JD: AI 智能体开发工程师

93.3%

检索准确率@3 (30 题评测)

28/30

引用来源准确 (真实)

40条

示例制度文档

企业问答答非所问、没出处。我在 enterprise-agent 里加了一套真实带数据的 RAG demo: 上传制度文档→提问→返回带条款引用的回答, 离线可跑、指标可量化。

业务痛点: 大模型直接回答企业制度/知识类问题易幻觉、无引用来源, 无法满足企业可信问答要求。

我的角色: 本人基于自建 enterprise-agent 的知识层 (ChromaDB + Dense+BM25 混合检索 RRF + Cross-Encoder 重排), 新增 examples/rag_demo 真实演示模块: 示例企业制度文档 + 可运行脚本 + 评测报告, 跑通「加载文档→建索引→提问→返回引用来源」全流程, 无 LLM Key 自动降级为 Top-K 检索结果。

方案架构

sample_policy.md (脱敏企业制度) → indexer.split_documents 语义切分 (chunk 500/overlap 50) → ChromaDB 建索引 → HybridRetriever (Dense+BM25→RRF) → Cross-Encoder 重排 (可选) → 带章节引用的回答; 无 Key 降级返回 Top-K 命中片段与来源。

工具栈

Python 3.11+

ChromaDB

rank-bm25

LangChain

RRF

Cross-Encoder (可选)

Ollama/OpenAI 兼容接口 (可选)

实施步骤

1. 编写脱敏公开企业制度示例文档 sample_policy.md (40 条制度)
2. 复用 knowledge 层实现 rag_demo.py: 加载→切分→建索引→混合检索→降级引用
3. 配置 6 条人工标注黄金条款的评测集, 内置 run_eval 量化指标
4. 本地实际运行 3-5 个示例提问并记录真实输出
5. 生成 RAG_Eval_Report.md: 检索准确率/拒答率/引用来源准确率
6. 编写 requirements.txt 与 README 说明运行方式

关键难点与解决

难点：中文制度条款检索命中率与无 Key 可演示性。解决：Dense+BM25 混合检索 + RRF 融合提升召回，Cross-Encoder 重排预留降级点；无 Key 时自动降级为 Top-K 片段+来源，保证离线可跑。

量化测算

检索准确率	@3: 93.3% (28/30, 30 题分层评测)
拒答率	16.7% (1/6, 检索层判定资料不足应拒答)
引用来源准确率	93.3% (28/30, 引用精确到《第X条》)
评测集	6 条人工标注黄金条款，指标可复现

可复用资产

- 示例企业制度文档（脱敏）
- RAG 演示脚本
- RAG 评测报告（真实运行数据）
- 运行说明 README

复盘

如果重做，本人会先把 Cross-Encoder 重排跑通并纳入评测对比（本机未装 sentence-transformers 已自动降级），并扩展评测集到 20+ 问，让检索准确率更稳健。

07 AI 评测与可观测性（评测集纳入 CI 回归 + OTel/Jaeger 全链路追踪）

[自建 Demo](#)

对应 JD：AI 智能体开发工程师

8

单会话Jaeger trace span

6条

CI每次push自动评测

≥60%

CI断言阈值（检索/引用）

智能体上线怕退化、出问题难定位。我把评测集纳入 GitHub Actions 每次 push 自动回归，并用 OpenTelemetry+Jaeger 打通 Router→Planner→Tool→Reviewer 全链路 trace，一次请求一条链、每阶段耗时可见。

业务痛点：智能体系统迭代缺少自动回归防线，评测依赖人工跑；生产环境一次会话跨多阶段（路由/规划/工具/评审），故障与慢节点无法按请求链路定位。

我的角色：本人为 enterprise-agent 补齐生产化双能力：① 将 examples/rag_demo 内置评测集纳入 CI（新增 eval-rag job），每次 push 自动跑 6 条评测并断言阈值、上传指标报告产物；② 基于 OpenTelemetry 实现全链路观测（observability 模块），对 Router/Planner/Retrieve/Tool/Reviewer/Respond 各节点打点，docker-compose 内置 Jaeger，真实运行会话抓取 Jaeger trace 截图与方案文档。

方案架构

评测回归：GitHub Actions（push 触发）→ uv 装依赖 → rag_demo.py --eval（无 Key 降级）→ 断言检索准确率≥60%/引用准确率≥60%/拒答率≤40% → upload-artifact 保存报告。可观测性：Agent 会话根 span（trace_id 贯穿）→ 各节点 span（记录 duration_ms）→ OTLP HTTP(4318) → Jaeger all-in-one（UI 16686）按 trace 聚合展示阶段耗时。

工具栈

Python 3.11+

OpenTelemetry SDK

OTLP HTTP/protobuf

Jaeger all-in-one

GitHub Actions CI

uv

FastAPI instrumentation

实施步骤

1. 评测集纳入 CI：新增 eval-rag job，py3.11/3.12 双矩阵，运行 rag_demo.py --eval --top-k 3 并断言阈值

- 2. 评测输出与本地命令完全一致，本地实际运行验证指标（检索准确率 93.3% / 拒答率 6.7% / 引用准确率 93.3%）
- 3. 实现 observability/telemetry.py: TracerProvider + OTLPSpanExporter + traced_node 节点打点装饰器
- 4. 编排层接入: nodes.py 全部节点 @traced_node 打点, graph.py 创建根 span 使 trace_id 贯穿整条链路
- 5. docker-compose 新增 Jaeger 服务, agent-api 配置 OTEL_EXPORTER_OTLP_ENDPOINT
- 6. 真实运行 run_trace_demo.py 会话经 OTLP 上报 Jaeger, 抓取 trace 详情页真实截图 (8 span 含 tool 循环)
- 7. 输出 docs/observability.md 方案文档 (架构/span 定义/trace_id 机制/复现步骤/真实截图)

关键难点与解决

难点: 评测 stdout 含 banner 非纯 JSON、OTLP 上报与 Jaeger 落地验证、trace_id 跨异步节点贯穿。解决: CI 断言脚本做 JSON 块提取; 本地起 Jaeger all-in-one 实际接收 OTLP 并查询 trace 验证; 根 span 包裹编排、子 span 经 Context 继承 trace_id (asyncio task 亦继承)。

量化测算

CI 回归	每次 push 自动跑 6 条评测, 断言检索≥60%/引用≥60%/拒答≤40%, 报告作为 artifact 保存
评测基线	top_k=3: 检索准确率 93.3%、拒答率 6.7%、引用来源准确率 93.3%
Jaeger trace	一次会话 8 个 span (根 + Router/Planner×2(工具循环)/Retrieve/Tool/Reviewer/Respond), trace_id 贯穿

可复用资产

OTel 观测模块 (telemetry.py + 节点打点)

Jaeger 服务编排 (docker-compose)

可观测性方案文档

Jaeger trace 真实截图

CI 评测回归 job

复盘

如果重做, 本人会为评测集补充检索延迟/LLM 回答质量维度并扩展 20+ 条, 同时把 Jaeger 接入 FastAPI 请求中间件实现线上请求级追踪, 形成评测-监控闭环。

08 办公流程自动化（Open Claw + Python 脚本）

[自建 Demo](#)

对应 JD：AI workflow落地工程师 / FDE / AI 项目负责人

自动化

跨系统重复流程脚本执行

兜底

异常告警+人工复核

SOP

流程与运维文档

采集、清洗、汇总这些重复活太吃人。我用Open Claw+Python脚本自动跑起来，异常再交人复核，稳且可维护。

业务痛点：信息采集、文档处理、数据清洗、报表与对账汇总等跨系统重复操作靠人工，慢且易错。

我的角色：本人部署调试 Open Claw 自动化 workflow 框架，并用 Python 编写自动化脚本批量完成文档处理与数据清洗，配合 Coze 编排，把重复流程改为自动执行 + 异常人工复核（影刀/钉钉 RPA 为方法储备）。

方案架构

触发（定时/事件）→ Open Claw / Python 自动化脚本（信息采集 / 文档处理 / 数据清洗 / 汇总）→ 大模型增强 → 结果汇总 → 异常告警 → 人工复核。

工具栈

Open Claw

Python

大模型 API

Coze

定时任务/脚本

影刀/钉钉 RPA（方法了解）

实施步骤

1. 盘点高频重复的跨系统操作
2. 设计自动化流程（触发→执行→汇总→告警）
3. 用 Open Claw 编排 + Python 脚本实现
4. 加异常处理、失败重试与人工复核兜底
5. 试点 → 统计替代人工时长与错误率 → 推广
6. 输出 SOP 与运维文档

关键难点与解决

难点：脚本在环境/界面变化时失效。解决：优先用 API 与结构化数据源、加输入容错与失败重试、异常告警与人工复核兜底，定期巡检。

量化测算

核心度量	替代人工时长 / 错误率下降
真实脚本	Python 批量文档处理与数据清洗（真实脚本）
风险降低	异常告警降低漏处理风险

* 测算模型 / 示例口径，实际以落地数据为准

可复用资产

- 自动化流程脚本（Open Claw/Python）
- 异常处理与告警配置
- SOP 与运维文档

复盘

如果重做，本人会优先选有 API 的系统做集成，把界面级自动化（RPA）用在确实无接口的场景，从根上提升稳定性。

09 Prompt 工程与评测体系 自建 Demo

对应 JD：所有 JD（尤其 workflow 落地、智能体开发）

A/B

对比测试驱动优化

Schema

JSON 输出强校验

模板化

Prompt 结构化沉淀

AI 输出像抽卡不稳。我用结构化 Prompt+JSON Schema 校验+评测集做 A/B，把抽卡变成可校验、可复现的生产线。

业务痛点：Prompt 靠感觉写，输出不稳定、不可校验，像「抽卡」，无法支撑生产环境。

我的角色：我建立 Prompt 库 + 评测集 + 输出校验规则 + 版本迭代记录，让 AI 输出稳定、可校验、可复现。

方案架构

Prompt 模板（分类/抽取/生成/审核/总结）→ JSON Schema/正则/规则校验 → 人工审核机制 → A/B 测试与准确率对比 → 版本迭代记录。

工具栈

大模型 API

JSON Schema

正则

Python

评测脚本

Coze/Dify

实施步骤

1. 按场景沉淀 Prompt 模板（分类/抽取/生成/审核/总结）
2. 定义 JSON Schema 与正则校验规则
3. 建立评测集（标注样本）与准确率基线
4. 做 A/B 测试对比不同 Prompt 效果
5. 建立人工审核与反馈闭环
6. 版本化管理 Prompt 与迭代记录

关键难点与解决

难点：业务场景多样、Prompt 难复用。解决：抽象出「角色+任务+约束+输出格式+示例」的结构化模板，用 Schema 强约束输出，让 Prompt 从「手艺」变为「工程」。

量化测算

核心度量	评测集准确率对比
输出质量	输出格式合规率提升
持续优化	A/B 测试驱动持续优化

* 测算模型 / 示例口径，实际以落地数据为准

可复用资产

- Prompt 模板库
- 评测集与标注样本
- JSON Schema/校验规则
- 版本迭代记录

复盘

如果重做，我会把评测自动化进 CI，每次 Prompt 变更自动跑评测集，防止「改好一个场景、改坏另一个」。

10 培训 / SOP / 推广运营包 已交付

对应 JD：AI 工作流落地工程师 / AI 项目负责人

70%

自助解决率目标

闭环

反馈收集→处理→迭代

SOP

带教培训体系

方案做出来客户不会用等于零。我做操作手册、培训脚本、FAQ和反馈闭环，把技术翻译成大白话——这就是FDE打通最后一公里。

业务痛点：AI 工作流搭好了，但业务人员不会用、不愿用，「AI 不好用」的反馈无人处理，采纳率低。

我的角色：本人面向客户与内部业务团队开展政策解读、申报流程与系统操作培训，制作操作手册、培训脚本与 FAQ，并参与内部技术培训体系搭建、部门工作流程与质量标准制定；以采纳率跟踪推广效果——这正是 FDE 打通「最后一公里」的核心能力。

方案架构

操作手册 → 培训视频 → FAQ → 反馈收集 → 「AI 不好用」处理流程 → 迭代优化 → 采纳率看板；带教 SOP 培养内部专员。

工具栈

操作手册

录屏工具

FAQ 知识库

钉钉/飞书

采纳率看板

实施步骤

1. 编写业务人员视角的操作手册（图解+步骤）
2. 录制培训视频脚本与常见问题 FAQ
3. 建立「AI 不好用」反馈收集与处理流程
4. 带教业务骨干与客户团队，培养内部推广力量
5. 用采纳率看板跟踪使用率与自助解决率
6. 按反馈迭代工作流与培训内容

关键难点与解决

难点：业务人员抵触新工具。**解决：**从「帮他们省事」的场景切入、先给甜头；培养内部专员做身边榜样；把反馈闭环可见化，让员工看到「提的问题真的被改了」。

量化测算

核心度量	培训覆盖率 / 采纳率
自助解决率	目标 ≥ 70%
闭环周期	反馈闭环周期缩短

* 测算模型 / 示例口径，实际以落地数据为准

可复用资产

- 操作手册
- 培训视频脚本
- FAQ 知识库
- 带教 SOP
- 采纳率看板

复盘

如果重做，我会把培训做成「场景化微课」而非长文档，并建立内部 AI 使用积分榜，用激励机制提升主动采纳率。

11 技术底座与运维证据

自建 Demo

对应 JD：AI 智能体开发工程师

Docker

Compose一键起服务

高可用

容错/重试/降级/熔断

监控

日志+告警+压测

现场环境受限还得稳。我用Docker Compose一键起服务、配降级/容错/熔断和健康检查，让系统在客户现场也跑得住。

业务痛点：AI 应用能跑起来，但在客户现场/资源受限环境不稳定、无监控、故障难排查，无法支撑高可用交付。

我的角色：本人在资源受限环境下用 Docker Compose 快速搭建并现场部署调试 enterprise-agent，做容器化部署、API 集成、容错重试降级、性能调优、故障排查与安全权限，让系统不仅「能搭」还能在客户现场「稳定跑起来」——这是 FDE 的现场交付与问题解决能力。

方案架构

Docker/K8s 部署 → API 集成 → 容错/重试/降级 → 监控与日志 → 性能调优与压测 → 安全与权限管理。

工具栈

Docker

K8s

FastAPI

Nginx

Prometheus

日志系统

Python/Java

实施步骤

1. 容器化打包（Docker）与编排（K8s）
2. 实现 API 集成与鉴权
3. 加入容错、重试、降级、熔断机制
4. 部署监控与日志，建立告警
5. 压测并发与响应，做性能调优
6. 输出部署文档、故障排查手册与安全权限方案

关键难点与解决

难点：大模型调用不稳定、超时与限流。解决：加重试与退避、多模型降级、结果缓存；用监控及时发现异常，用压测提前暴露瓶颈。

量化测算

核心度量	系统可用性 / 故障恢复时间
性能	并发量与响应时间压测
安全	安全权限合规

* 测算模型 / 示例口径，实际以落地数据为准

可复用资产

- Docker/K8s 部署配置
- 容错重试模块
- 监控与告警方案
- 部署与排查文档

复盘

如果重做，我会引入蓝绿/灰度发布，让模型或 Prompt 升级可平滑回滚，避免「一次更新全线故障」。

12 项目复盘与量化数据报告 已交付

对应 JD：所有 JD（JD 明确要求「必须有数据」）

300+

政府/企业项目交付

24家

单年高企通过（含复审）

100+

服务企业客户

做完说不清价值等于白干。我给每个项目建卡片、埋基线、前后对比出数据看板，300+项目、单年24家（含复审）高企就是这么量化出来的。

业务痛点：项目做完缺乏系统复盘，成果说不清、数据拿不出，无法证明价值、也无法沉淀可复用资产。

我的角色：我为每个项目建立复盘报告与数据看板，用统一口径回答「痛点—主导—工具—成效—资产」，把成果量化、把经验沉淀。

方案架构

项目卡片 → 数据看板 → 复盘 PPT；每个项目回答：业务痛点是什么 / 我主导了什么 / 用了什么工具 / 上线后省了多少人提了多少效 / 有什么可复用资产。

工具栈

数据看板

复盘 PPT

项目卡片模板

Excel/BI

实施步骤

1. 按统一模板沉淀每个项目卡片
2. 采集上线前后对比数据（处理量/人天/准确率/转化率/成本/周期）
3. 制作数据看板与复盘 PPT
4. 提炼可复用资产（Prompt/SOP/代码/模板/看板）
5. 归档并持续更新

关键难点与解决

难点：上线前缺乏基线数据，成效难对比。解决：项目启动即埋点采集基线，用「前后对比」而非「事后估算」；统一度量口径避免各说各话。

量化测算

核心度量	省多少人天 / 处理量 / 准确率 / 转化率 / 成本下降 / 周期缩短
方法	用前后对比数据证明价值

* 测算模型 / 示例口径，实际以落地数据为准

可复用资产

- 项目卡片模板
- 数据看板
- 复盘 PPT
- 可复用资产库

复盘

如果重做，我会把「基线采集」作为项目立项的强制前置动作，没有基线不启动，从根上保证成效可量化。

13 企业级智能体 MCP Server 三件套（开源）

已交

付

对应 JD：所有 JD（AI 工程化 / 智能体 / MCP 生态）

3个

MCP Server开源

35

pytest用例全通过

100%

用例通过率

把 3 个可独立复用的 MCP Server 开源到 GitHub：工具调度、作品集检索、RAG 知识问答，各带测试与文档，证明我不只会用工具，还会造工具。

业务痛点：企业接入 MCP 生态时缺现成、可测试、可维护的 Server 实现，团队只能从零造轮子，质量与进度都不可控。

我的角色：我基于 FastMCP 1.x 独立设计并实现三个 MCP Server（enterprise-agent-tools-mcp / portfolio-mcp / rag-knowledge-base-mcp），含 pyproject、README、测试与 uv.lock，统一放到 enterprise-agents 仓库 mcp/ 目录并开源。

方案架构

三个子项目各成独立 Python 包：enterprise-agent-tools-mcp 提供工具调度与统一入口；portfolio-mcp 提供作品集证据检索；rag-knowledge-base-mcp 提供中文 RAG 知识库问答（BM25 bigram）。每个含 tests/、README.md、pyproject.toml、uv.lock。

工具栈

Python

FastMCP 1.x

uv

pytest

GitHub

BM25（中文 bigram）

实施步骤

1. 搭建三子项目骨架（pyproject/README/tests）
2. 实现各自 MCP 工具与数据层
3. 编写 pytest 用例并跑通（10/9/16 passed）
4. 生成 uv.lock 并核对 .gitignore 无误伤
5. 推送 GitHub 并集成进作品集网页

关键难点与解决

难点：三项目共享仓库内目录隔离与依赖锁定，避免互相污染。解决：各自独立 pyproject + uv.lock，CI 只扫根目录测试路径，mcp/ 子项目自行保证可测可跑。

量化测算

测试	三子项目 pytest 35 个用例全部通过
依赖锁定	uv.lock 均存在且 .gitignore 无误伤
可访问	GitHub 已可公开访问

可复用资产

mcp/enterprise-agent-tools-mcp

mcp/portfolio-mcp

mcp/rag-knowledge-base-mcp

共享仓库 enterprise-agents

证明链接

github.com/Jav1es/enterprise-agents/tree/main/mcp

复盘

如果重做，我会在一开始就统一三项目的脚手架与版本策略，并为每个 Server 增加 schema 快照测试，进一步降低集成回归风险。

14 dsh-bridge: 让 Marvis 调用本机 DeepSeek Harness (开源)

开源工具

对应 JD: AI 智能体开发工程师 / AI 工作流落地工程师 / FDE

3

MCP 工具 (零依赖手写协议)

241×77

技能 × MCP 工具本地检索

10 秒

派工单生成耗时

两个 AI 助手各干各的，能力接不起来。我写了一个 MCP Server 把本机 DSH 接进 Marvis：先出派工单、再按建议调技能与工具，零依赖、可复现、CI 全绿。

业务痛点：本机同时跑着 Marvis (日常主力) 与 DeepSeek Harness (带完整工具链的重活代理)，但两者互不相通：Marvis 不知道 DSH 能做什么，DSH 也读不到 Marvis 的技能与 MCP 清单，只能手工在两个窗口之间搬运内容。

我的角色：我独立完成了整个桥接的设计与实现：先用文件系统与 SQLite 把 Marvis 的技能库 (241 个) 与 MCP 目录 (服务器与工具) 读出来，做本地关键词检索挑候选，再让 DSH 生成结构化派工单。协议层是手写的 JSON-RPC (零运行时依赖)，另配双语文档、排错手册与 CI。

方案架构

Marvis (MCP 客户端) → dsh-bridge MCP Server (stdio, 换行分隔 JSON-RPC, 零依赖) → 本地检索 (读 ~/.marvis/skills/*/SKILL.md 的 frontmatter + data.db 的 mcp_server_index / mcp_tool_index, 仅取 enabled=1) → dsh --profile headless 生成派工单 → 回传 Marvis 按「建议调用」执行。任务文本经临时文件传递，规避命令行转义与中文编码问题。

工具栈

Node.js (零依赖, 仅内置模块)

MCP 协议 (stdio JSON-RPC 2.0)

node:sqlite

PowerShell 7

DeepSeek Harness headless

GitHub Actions CI

实施步骤

1. 确认 DSH 的对外调用面：headless 一次性执行可用，ACP/SDK 内置；确认它只有 MCP 客户端、不能当 MCP 服务器
2. 修复 headless profile 凭据路由，使 dsh --profile headless 真正可用
3. 手写 MCP stdio 服务器：实现 initialize / tools/list / tools/call / ping, stdout 只走协议、日志走 stderr

- 4. 解决中文与转义：任务文本写临时文件、由临时 .ps1 以变量传参，绝不做命令行字符串拼接
- 5. 解决进程树泄漏：超时用 taskkill /T /F 杀整棵进程树，不留野进程
- 6. 解决精简 PATH：pwsh 与 dsh 均按绝对路径回退，实测 PATH 只剩 node 时仍可用
- 7. 新增 dsh_plan：读 Marvis 技能与 MCP 目录 → 本地中英混排分词检索 → 只把 top-K 喂给模型生成六节派工单
- 8. 加约束消除幻觉：规划器不得读文件或调工具（76 秒 → 10 秒）；工具名必须逐字取自候选清单
- 9. 补 CI：文件布局、Node 与 PowerShell 语法、MCP 协议冒烟、敏感信息扫描，五步全绿
- 10. 补双语文档：设计取舍、MCP 参考、排错手册（含 Marvis 导入技能的四条硬性要求）

关键难点与解决

难点有三个。一是 DSH 不能当 MCP 服务器——社区 4382 条插件目录里，101 个 MCP 相关插件全是「DSH 去连别人」，没有「把 DSH 暴露出去」，只能自己补一层包装。二是模型会臆造候选清单外的工具名，调用方照着调就会失败，用提示词硬约束解决，复测越界为 0。三是规划器会「顺手把活干了」，单次调用 76 秒，加「只做规划」约束后降到 10 秒。

量化测算

MCP 工具	3 个 (dsh_plan / dsh_run / dsh_status)，零运行时依赖
本地检索	覆盖 241 个技能 × 77 个 MCP 工具，skip_llm 模式 0.1 秒返回
派工单	完整生成约 10 秒（优化前 76 秒）
Skill 封装	6/6 用例通过（中文、管道、JSON 事件流、空任务、工作目录、多行引号）
端到端实战	对三份简历做独立审计，148 秒产出 622 行报告，挑出 3 处高风险问题
CI	五步全绿：布局 / Node 语法 / PowerShell 语法 / MCP 冒烟 / 敏感信息扫描

可复用资产

零依赖 MCP stdio 服务器 server.mjs

一次性 dsh 执行器 dsh-runner.ps1

Marvis 技能包 (SKILL.md + dsh_ask.ps1)

双语文档 (README.md / README.en.md)

设计取舍 docs/design.md

MCP 参考 docs/mcp-reference.md

排错手册 docs/troubleshooting.md

CI 工作流 .github/workflows/ci.yml

原创图标 (鲸鱼 × 桥)

证明链接

github.com/Jav1es/dsh-bridge

设计取舍

MCP 参考

排错手册

复盘

如果重做，我会一开始就把「调用方能不能真的调到」当成硬需求：第一版既没过滤 enabled，也没约束模型只能用清单内的名字，等于把失败推给了下一环。另外我低估了「规划器会顺手干活」的惯性——提示词里少一句「只做规划」，单次调用就贵了 7 倍。

15 DeepSeek Harness 本机插件三件套（开源）

个人开源

对应 JD：AI 智能体开发工程师 / 平台工程

3个

自研 DSH 插件

100%

离线可跑 · 零云依赖

MIT

开源许可

AI 助手天天用，但工具链缺什么就得等官方排期。我直接读了宿主插件的源码与打包格式，自己写了三个插件补上空白：消息推手机、文件能预览、待办与推理过程看得见。

业务痛点：日常重度使用 DeepSeek Harness，但官方生态里缺三样东西：长任务跑完人不在电脑前就不知道结果；侧边栏点开图片和 PDF 只能看文件名；待办清单与模型的推理过程藏在会话流里，没有独立面板可看。这些不是「不会用」，而是工具本身没有。

我的角色：我独立完成了三个插件的设计、实现与打包。做法是先逆向宿主：读 app.asar 里的插件加载器与已有插件源码，确认插件的注册点（工具注册、侧边栏文件查看器注册、面板插槽）与 bundle 打包约定，再用 ESM 写实现、按约定挂载。全过程不需要官方 SDK，也不依赖任何云端服务。

方案架构

宿主 DeepSeek Harness → profile 的 bundles 清单加载插件 → 插件通过 package.json 的 dsh.bundle.patch 声明配置层 → cordis.patch.yml 注入注册点。三个插件分别是：host 侧插件（监听助手回复事件 → 经 Server酱 HTTP 接口推送到微信）、web 侧插件 A（向 better-sidebar 注册图片/PDF 文件查看器）、web 侧插件 B（注册右侧栏 Todo 面板与 Thinking 面板两个标签页）。

工具栈

Node.js ESM

零运行时依赖

DSH 插件 API

Cordis bundle

React

app.asar 逆向

实施步骤

1. 逆向宿主：解包 app.asar，读插件加载器与 dsh-tool-todo 等已有插件
2. 确认注册点签名：工具注册、betterSidebar.registerFileViewer、右侧栏面板插槽
3. 按 bundle 约定搭建骨架：package.json 声明 dsh.bundle.patch，cordis.patch.yml 声明配置
4. 实现消息推送插件：未配置密钥时静默停用，不报错
5. 实现文件预览插件：注册图片与 PDF 查看器
6. 实现面板插件：Todo 面板读待办投影，Thinking 面板展示推理流

- 7. 凭据卫生：密钥只从本机配置读取，代码里默认为空串
- 8. 纳入版本控制并开源：抽独立仓库、补 README 与 MIT、推送前跑凭据扫描

关键难点与解决

难点在于没有任何官方文档可查，全部要靠读源码反推。最典型的是插件加载时机与注册点的签名——写错了不会报错，只是「没反应」，只能靠对照已有插件的写法逐个比对。另外消息推送插件踩到凭据卫生的坑：密钥如果写进代码就会随仓库泄露，所以设计成默认空值 + 本机配置文件读取，未配置时静默降级。

量化测算

自研 DSH 插件	3 个，全部 MIT 开源
运行时依赖	零，仅用 Node 内置模块与宿主 API
推送密钥	默认空值，未配置时静默停用（0 泄露风险）
凭据扫描	全仓库 14 个文件，命中 0 处真实密钥

可复用资产

dsh-serverchan-notify

dsh-sidebar-preview

dsh-workpanel

仓库 README

MIT 许可

证明链接

github.com/Jav1es/dsh-local-plugins

复盘

如果重做，我会先把「宿主怎么加载插件」写成一页笔记再动手——中途有两次因为注册点签名理解错了，代码不报错但功能不生效。另外这三个插件一开始直接写在插件目录里、完全没进版本控制，等于单点存放；直到做资产盘点才发现，这也是我现在坚持「凡自产先入库」的原因。

16 DSH 运维工具箱：升级自检 / 插件兼容预检 / 生态全量调研（开源）

个人开源

对应 JD：AI 智能体开发工程师 / 平台工程

4382

插件目录全量调研

3

工具与文档组件

MIT

开源许可

一次升级把已装插件静默打废，排查半天。我干脆把「升级自检、装前兼容预检、缺失能力补丁」做成可跑的脚本与文档开源出来——这类坑不该靠人肉记。

业务痛点：DSH 桌面端自动升级后，已装插件会因 peer 依赖精确 pin 与运行时失配而「看着装上了、功能却不生效」；社区工具的兼容性口径与宿主真实安装门不一致（前者偏乐观），照着它选型会踩坑。另外生态里缺少的能力只能自己补，而补丁一升级就被覆盖。

我的角色：我独立完成了工具链的设计、实现与文档化：把升级自检、插件兼容性预检、缺失能力补丁固化成可重复执行的脚本；对社区插件目录做全量兼容性调研并写出选型报告；所有机器相关路径改为环境变量或按当前用户推导，脚本对锚点做唯一性检查，绝不静默改错。

方案架构

三个组件各自独立可跑：dsh-maintenance（比对桌面运行时与全局 CLI 版本、核验 peer 兼容性，退出码 0/1/3 可做门禁，含 cost-meter 官方余额的账号登录态补丁与 app.asar 只读探针）；compat-check（纯 PowerShell 实现 semver 判定，判定候选插件 peer 与 engines）；ecosystem-survey（对 4382 条社区插件目录做全量兼容性判定与选型分析）。

工具栈

PowerShell 5.1+

Node.js

semver 自实现

app.asar 逆向

MCP studio

实施步骤

1. 复盘真实事故：桌面端 rc.1 升到 rc.2，某插件 peer 是精确 pin 被拒绝加载
2. 实现升级自检脚本，退出码三分档，对已知假阳性单独说明
3. 实现兼容性预检工具：纯 PowerShell 写 semver 判定，对照 node-semver 验证
4. 定位 cost-meter 余额为空的根因：只实现了 API Key 取数路径，本机是账号登录态
5. 写补丁增加账号 token 回退分支，令牌只在签发来源与目标 origin 一致时返回
6. 补幂等打补丁 / 回滚 / 端到端验证三个脚本，结构一变就 FATAL 退出

- 7. 做 4382 条插件目录全量调研，区分「社区口径」与「宿主真实安装门」
- 8. 脱敏后开源：机器路径改为环境变量，实跑验证没有改坏

关键难点与解决

难点有三处。一是口径不一致——社区市场工具判定「兼容」的 481 个包，按宿主真实安装门其实是会被拒的，所以必须自己有一把尺子并把两套口径都写清楚。二是补丁的健壮性——插件升级会覆盖源码，补丁必须幂等、必须对锚点做唯一性检查。三是脱敏——脚本原本写死了本机用户名路径，开源前必须全部改为环境变量，改完还要逐个实跑验证。

量化测算

插件目录调研	4382 条：2253 条有 npm 包可探测，2129 条无 npm 包
兼容性判定	可安装 604 / 未声明版本可放行 918 / 被明确阻断 731
口径偏差发现	社区市场判「兼容」但被宿主真实安装门阻断的达 481 条
升级自检退出码	0 通过 / 1 不兼容 / 3 无法核实（不得当通过）
凭据扫描	全仓库 14 个文件命中 0 处真实密钥

可复用资产

升级后自检脚本

DSH 维护手册

插件兼容性预检工具

cost-meter 补丁（三脚本）

app.asar 只读探针

生态选型报告 + xlsx

证明链接

github.com/Jav1es/dsh-toolkit

复盘

如果重做，我会把「升级自检」挂到桌面端升级流程里自动触发，而不是靠人记得跑——这次事故的本质就是「没有门禁」。另外我一开始直接采信了社区市场工具的兼容性判定，直到实测发现它的口径比宿主真实安装门宽松得多，才回头按宿主规则重算；这 481 条的差值就是「不自己验证口径」的代价。

17 知识指南：把「先检索再作答」写成可复用的规程（开源）

个人开源

对应 JD：AI 智能体开发工程师

1

可复用检索规程

100%

零云依赖 · 纯 Markdown

MIT

开源许可

AI 助手最大的坑不是答不好，是凭记忆编。这个 skill 把「先检索再作答、强制标来源、置信度标注、未命中先做正对照」写成一条条可执行的规程，让答案可验证、让「查不到」变可信。

业务痛点：DSH 里没有自动上下文检索——除了 AGENTS.md，一切知识都得主动调工具才会进上下文。同一类问题在不同会话里会得到不同答案：有时认真检索、有时凭训练记忆直接编。更麻烦的是「查不到」本身不可信——检索通道故障和内容真的不存在，两种情况都表现为搜不到，一旦分不清就会把「没查」误当成「没有」，写进对外材料就是事故。

我的角色：我独立完成了规程的设计、落地与验证。先把「DSH 到底有没有自动检索」这件事查实（结论是没有），再把散落的检索经验收敛成有路由、有格式、有红线、有坑位清单的规程；然后用三道真实工作题做能力测试（新会话、无上下文提示），验证纪律是真的被执行、而不是只写在文件里。

方案架构

规程分三步：① 问题类型判别——本地事实 / 外部现状 / 深度调研 / 推理写作，本地优先于联网；② 检索路由——本地走文件与 TiddlyWiki（长期记忆），外部走 AnySearch，垂直库优先（学术 / 代码 / 工商 / 财报 / 专利 / 法律 / 安全）；③ 回答格式——结论先行，每个关键事实标来源，末尾标证据强度（高 / 中 / 低）。载体是单文件 SKILL.md，按 frontmatter 的 name + description 自动路由装载。

工具栈

纯 Markdown

无构建无依赖

DSH skill 机制

AnySearch

TiddlyWiki

实施步骤

1. 查实前提：确认 DSH 除 AGENTS.md 外没有自动上下文检索
2. 勘测本机 knowledge 库实况：18 张表但 0 篇文档、插件未安装，据此排除依赖它的方案
3. 设计问题类型判别表，并写死例外（纯概念解释、纯代码任务、就聊聊）
4. 设计检索路由：本地文件 + TiddlyWiki 双路，外部走 AnySearch；本地优先于联网
5. 写回答格式与强制规则：来源标注、禁止臆造、不足就说不足、时效性标注

- 6. 写正对照纪律：任何「不存在 / 没有」的结论，必须先验证检索通道本身有效
- 7. 沉淀本机坑位清单：双实例崩溃、web seam 无兜底链、换模型代价、误切昂贵档位
- 8. 能力测试：新开会话不加提示跑三道真实工作题，验证纪律真实执行
- 9. 按 frontmatter 白名单校准 name 与 description——缺一即静默失效，装完实调验收
- 10. 脱敏后开源：去掉机器特定路径，扫凭据，MIT 许可

关键难点与解决

难点不在写字，在三处判断。一是「什么时候不该检索」——把规则写成「凡事必查」会导致大量无意义的调用，比不检索更糟，所以必须写死例外并给出判据。二是「查不到」这件事的可信度：检索通道故障与内容不存在在表面上完全一样，如果不强制做正对照，就会把「我没查到」写成「它不存在」，这类错误一旦进入对外材料就是硬伤。三是诚实披露——本机知识库是空的，如果按常规叙事写成「知识库 / RAG」就是吹牛，所以明确走文件 + TiddlyWiki + AnySearch 三路。

量化测算

规程规模	单文件 SKILL.md，正文 95 行，零构建零依赖
检索三步闭环	问题类型判别 → 检索路由 → 回答格式（来源 + 置信度）
外部检索通道	AnySearch，7 类垂直库
正对照纪律	1 条：任何「不存在」结论强制先验证通道有效性
能力测试	新会话不加提示跑 3 道真实工作题，纪律真实执行

可复用资产

- SKILL.md 规程全文
- 本机坑位清单
- 成本纪律三条
- README（含诚实披露）
- MIT 许可

证明链接

github.com/Jav1es/dsh-knowledge-guide

复盘

如果重做，我会把「本地知识路由表」从 SKILL.md 里抽成独立文件单独维护——现在它和规程写在一起，改了纪律、路由表却要跟着改，两边耦合容易各自漂移。另外这一条规程的价值恰恰在于它「没什么技术含量」：真正的难点一直是逼自己在每个会话里都老实执行，而这恰恰是最容易偷懒的地方——把它固化成 skill 至少让偷懒变成了显式的偏离，而不是无声的省略。

陆 行业定制规划能力 · 行业定制样本

体系化思维 + 行业理解 + 信息化蓝图

除通用作品集外，我针对具体企业做过深度行业定制规划，证明「体系化思维 + 行业理解 + 信息化蓝图」能力。

✓ 真实完成的作品样本

某高分子新材料上市企业 · 信息化总体规划与数字化转型方案（含 AI 赋能）

面向某高分子新材料上市企业（建筑密封胶行业，国标起草单位，客户名称已脱敏）编制的公司级信息化总体规划。以「入职后规划」视角，覆盖 ERP / SRM / PLM / MES / HR / LIMS 五大业务系统 + 数据中台 + AI 能力平台 + 基础设施安全，并深度融入 AIGC 与 AI Agent 落地场景。

三大能力亮点

能力	说明
体系化规划	L1-L5 成熟度模型 + 总体蓝图 + 按系统完善程度推进的优先级矩阵
行业深度适配	配方研发 Copilot、多标准合规核查、批次质量追溯等 8 个专属 AI 场景
财务方向强项	业财一体化、研发费用归集与加计扣除，契合高新技术企业真实需求

柒 现场可演示清单

可当场演示、或按需提供录屏 / 脱敏源文件的内容，证明「能落地、跑得起来」

演示项	内容	状态
enterprise-agent 智能体	本地 Docker Compose 一键起服务，现场演示 workflow 编排 / RAG 问答 / 工具调用 / 容错降级	可现场演示
企业问答 / 文档 AIGC Demo	Coze + 多维表「AI 先做、人只审」双层 workflow，现场演示问答与文档生成	自建 demo 可演示
项目监测报告	自建业务数据库 + SQL 报表，现场打开《项目监测报告》看数据如何驱动决策	真实产出可展示
某高分子新材料企业规划书	完整信息化与 AI 规划：成熟度模型、蓝图、业财一体化、8 个 AI 场景	已完成 · 可发送脱敏版

捌 诚信说明与口径

真实项

- 「企业级智能体系统 enterprise-agent」为本人本地开发工程（含完整开发文档，可现场演示，已开源至 GitHub）。
- 《某高分子新材料企业规划方案》、高企认定 / 项目申报交付、自建项目信息数据库与《项目监测报告》为本人客户服务真实经历。
- 各模块「证明链接」多为本人制作的实物材料。

口径说明

- 凡成效类数字均标注为「测算模型 / 示例口径」，实际以落地数据为准。
- 电商相关场景为自建 demo / 方法储备，非电商客户实绩。
- FDE 板块两则实战故事基于真实项目，企业名与年度数据面试时可脱敏提供。

关键数字口径（统一）

项	口径
工作年限	5 年+
项目数	300+（其中申报并通过 200+）
服务企业	100+
高企通过	24 家（含复审）
提效	约 40%

联系方式

在线作品集：jav1es.github.io/portfolio | GitHub：github.com/Jav1es | 联系方式：面试时提供